

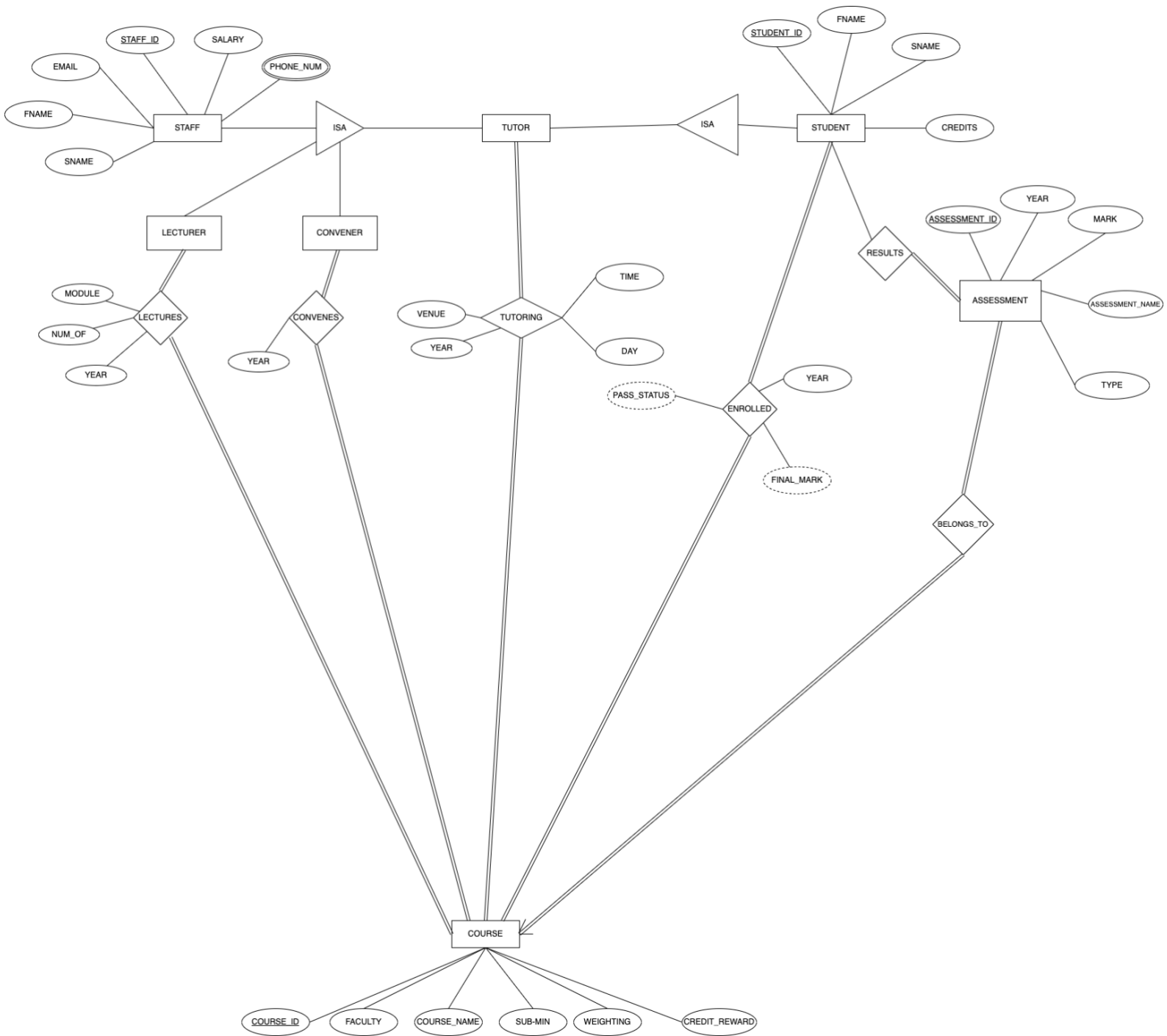
CSC2001F Assignment 2

By: CRSALE010, CRSNIC014, ERSJAC011, IFRFAB001, MCKCON007

Group 26

2023

Question A



Question B

Staff (Staff_ID, Fname, Sname, Email, Salary)

StaffPhoneNums (Staff_ID, phoneNum)

Lectures (Staff_ID, Course_ID, Module, Num_Of, Year)

Convenes (Staff_ID, Course_ID, Year)

Tutoring (Staff_ID, Student_ID, Course_ID, Venue, Time, Day, Year)

Student (Student_ID, Fname, Sname, Credits)

Assessment (Assessment_ID, Year, Mark, Assessment_Name, Type, CourseID, StudentID)

Enrolled (Student_ID, Course_ID, Year)

Course (Course_ID, Faculty, Course_Name, Sub-Min, Weighting, Credit_Reward)

Question C

The query is:

```
SELECT
    email,
    COUNT(*) AS "numberOfSalesMade",
    SUM(quantityOrdered) AS "numItemsSold",
    ROUND(SUM(quantityOrdered * priceEach), 0) AS "revenueMade"
FROM (
    orders
    NATURAL JOIN orderdetails
)
NATURAL JOIN customers
INNER JOIN employees ON customers.salesRepEmployeeNumber = employees.employeeNumber
GROUP BY
    email
HAVING COUNT(*) = (
    SELECT
        MAX(count.counter)
    FROM (
        SELECT
            COUNT(*) AS counter
        FROM orders
        NATURAL JOIN orderdetails
        NATURAL JOIN customers
        GROUP BY
            customers.salesRepEmployeeNumber
        ) AS count
    );
```

The output from the query is:

	email varchar	numberOfSalesMade bigint	numItemsSold newdecimal	revenueMade double
1	ghernande@classicmodelcars.com	396	14231	1258578

This query serves to display the email address, the number of sales made, the number of items they have sold (there can be multiple items in one order) and the total money they made with these sales of the salesRepEmployee that has made the most sales of items, not orders.

The tables joined in the query:

Orders is NATURAL JOINed with orderdetails to give all the information about one order by one customer (returns a new table (table 1) where orderNumbers are the same)

This results in a table with a snippet as shown (only showing relevant fields)

	Q	orderNumber int	productCode varchar	salesRepEmployeeNu int	quantityOrdered int	priceEach double
1		10100	S18_1749	1216	30	136
2		10100	S18_2248	1216	50	55.09
3		10100	S18_4409	1216	22	75.46
4		10100	S24_3969	1216	49	35.29
5		10101	S18_2325	1504	25	108.06
6		10101	S18_2795	1504	26	167.06
7		10101	S24_1937	1504	45	32.53
8		10101	S24_2022	1504	46	44.35
9		10102	S18_1342	1286	39	95.55
10		10102	S18_1367	1286	41	43.13
11		10103	S10_1949	1504	26	214.3
12		10103	S10_4962	1504	42	119.67
13		10103	S12_1666	1504	27	121.64
14		10103	S18_1097	1504	35	94.5
15		10103	S18_2432	1504	22	58.34
16		10103	S18_2949	1504	27	92.19

Note that orderdetails contains multiple products for one order, so a join is needed to ensure we count every product that was ordered rather than the number of orders and to use the quantityOrdered and priceEach attributes

Then we NATURAL JOIN with the customers table (with table 1 created in prior NATURAL JOIN) to create a new table (table 2) which gives all results where the customer number is the same. Each customer is associated with one salesRepEmployee, given in the customers table. To get the salesRepEmployeeNumber for an order, we thus join the customers table to the table 1 (that gives the salesRepEmployee shown in the above image)

We then inner join table 2 with employees to get the email address associated with one salesRepEmployee. This is joined on the salesRepEmployeeNumber and the employeeNumber

The SELECT then uses all the information in these tables to get:

The aggregate function COUNT(*) counts the number of rows associated with the sales made by a salesRepEmployees (a sale being the purchase of an item, not placing an order)

The aggregate function SUM(quantityOrdered) sums up all the values in the quantityOrdered that is associated with each salesRepEmployeeNumber. This is thus the number of items a sales employee has sold

The aggregate function ROUND(SUM(quantityOrdered * priceEach), 0) sums up all the monetary values associated with each order, then round it to nearest whole number

The aggregate functions are dealt with by a GROUP BY which then groups the table into sections based on the email (which is unique for each salesRepEmployee) so indirectly grouping by salesRepEmployeeNumber without causing errors

The HAVING condition involves a nested select that we will deal with in 2 parts:

1. The nested select uses another nested select to create and use a new table to get the data from

The table the data is from is the table that results from:

```
SELECT
  COUNT(*) AS counter
FROM orders
  NATURAL JOIN orderdetails
  NATURAL JOIN customers
GROUP BY
  customers.salesRepEmployeeNumber
```

Note that we rename the COUNT(*) attribute to "counter". This will be used later to access this variable. The joins follow similarly from above and this counts how many times a salesRepEmployee has made a sale of an item. The table for this query looks as such:

	Q	counter bigint
1		152
2		220
3		142
4		396
5		272
6		211
7		137
8		236
9		331
10		177
11		114
12		185
13		114
14		185
15		124

The "AS count" after the brackets serves to rename this table to "count"

2. The statement SELECT MAX(count.counter) thus gets the max counter from this count table

So we then choose all salesRepEmployees that have the same maximum counter, with HAVING COUNT(*) = (<the max counter from the inner select query>)

Question D

During the planning phase, each member got involved and gave their opinion on what we should do at each point. Everyone contributed to the planning, which was the bulk of the work as we planned possible routes for the questions. After this, MCKCON007 and IFRFAB001 used this planning to complete Question A and CRSALE010, CRSNIC014 and ERSJAC011 completed Question B and C. Each group then checked the other groups work to discuss possible changes.

Question E

Completed via Vula